

E-COMMERCE ORDER MANAGEMENT SYSTEM

PROJECT TITLE

E-Commerce Order Management System

OBJECTIVE

The objective of this project is to develop a console-based E-Commerce Order Management System that allows users to manage products, register customers, place orders, process payments, and track order status using Java and Object-Oriented Programming principles.

TECHNOLOGIES USED

- Java 21
- Standard Java SE Libraries (UUID, List, ArrayList, LocalDateTime)
- IntelliJ IDEA
- Object-Oriented Programming (OOP)
- Git & GitHub

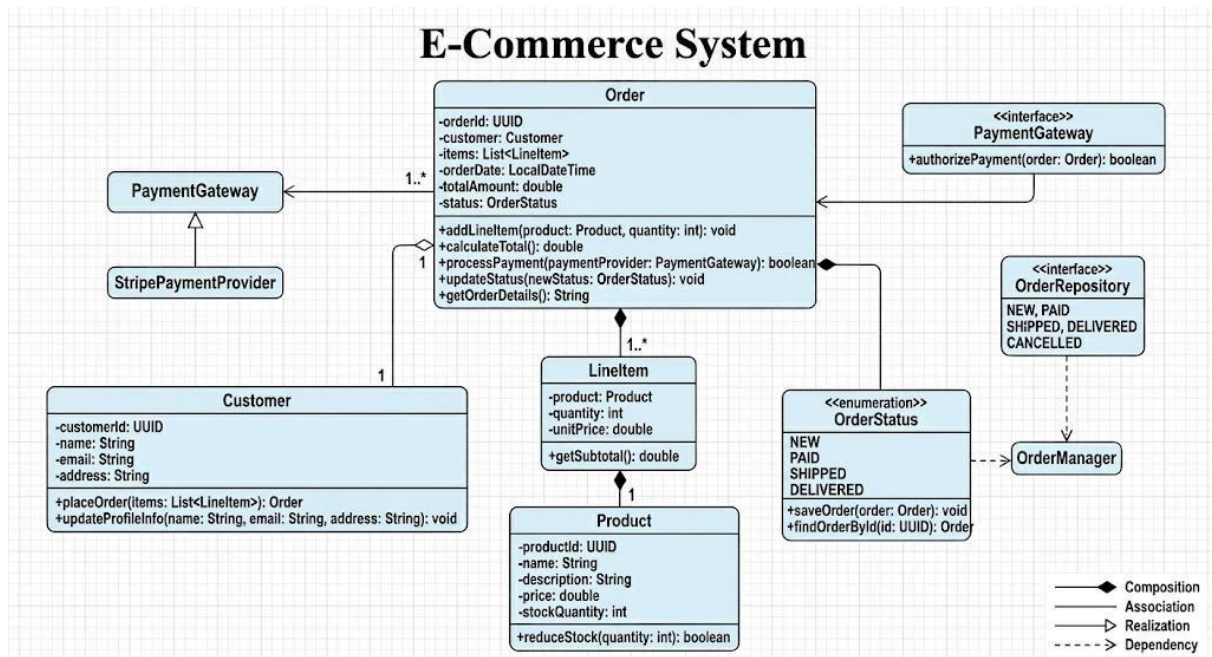
PROJECT DESCRIPTION

The E-Commerce Order Management System is a console-based application developed using core Java. The system manages the full lifecycle of an order — from product catalogue management and customer registration to order placement, Stripe payment simulation, and order status progression. All data is stored in-memory using a HashMap-backed repository, making the system fully self-contained and runnable without any external database setup. The application provides a menu-driven interface for all operations.

FEATURES

- Update Order Status (NEW → PAID → SHIPPED → DELIVERED)
- Order Cancellation
- View All Orders Report
- Filter Orders by Status
- Order Summary with Grand Total

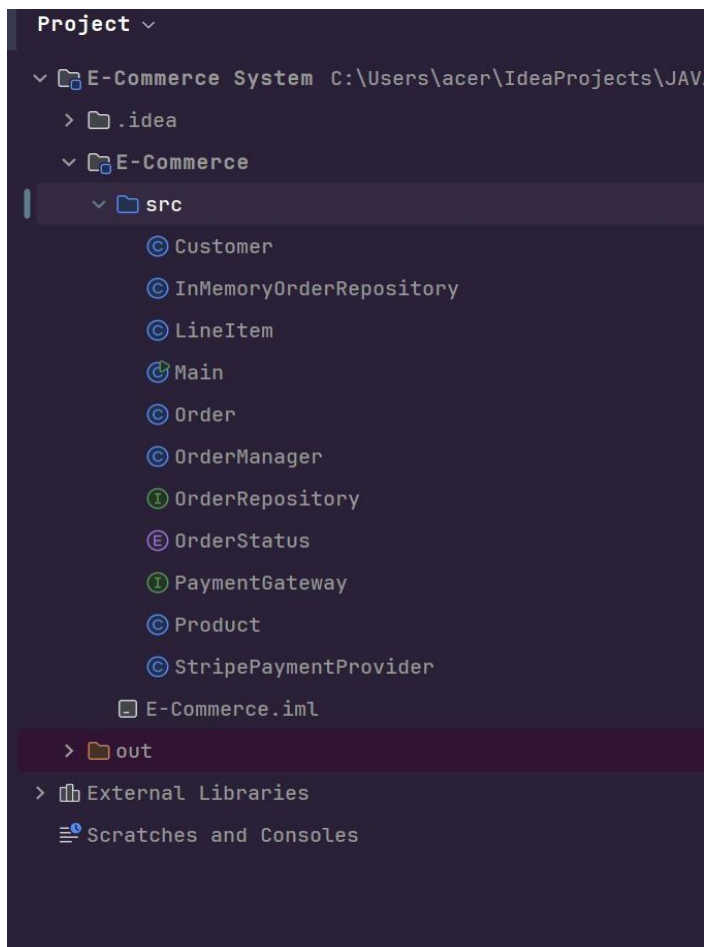
SYSTEM DESIGN — UML CLASS DIAGRAM



Relationships:

- Order has a Composition relationship with LineItem (1..*)
- Customer has an Association relationship with Order (1..*)
- LineItem has an Association relationship with Product (1)
- StripePaymentProvider realizes the PaymentGateway interface
- InMemoryOrderRepository realizes the OrderRepository interface
- OrderManager depends on OrderRepository (Dependency)
- OrderStatus is an Enumeration used by Order

PROJECT STRUCTURE



CLASS DESCRIPTIONS

- *OrderStatus.java* — Enumeration defining the five order lifecycle states: NEW, PAID, SHIPPED, DELIVERED, CANCELLED
- *Product.java* — Represents a product in the catalogue with fields productId, name, description, price, and stockQuantity. Contains the reduceStock() method with validation.
- *LineItem.java* — Represents a single item inside an order. Snapshots the unit price at creation time so future product price changes do not affect existing orders.
- *Customer.java* — Represents a registered customer. Contains the placeOrder() method which reduces product stock and creates the Order object.
- *Order.java* — Core class managing the order lifecycle. Contains methods for adding line items, calculating totals, processing payment, and updating status with transition guards.
- *PaymentGateway.java* — Interface defining the authorizePayment() contract for any payment provider.

- *StripePaymentProvider.java* — Concrete implementation of *PaymentGateway* simulating Stripe payment authorization with decline rules for zero amount and exceeding charge limits.
- *OrderRepository.java* — Interface defining the persistence contract: *saveOrder*, *findOrderById*, *findAllOrders*, *deleteOrder*.
- *InMemoryOrderRepository.java* — *HashMap*-backed implementation of *OrderRepository* for fully self-contained in-memory storage.
- *OrderManager.java* — Central service facade coordinating all order operations: create, pay, update status, cancel, retrieve, and report.
- *Main.java* — Entry point providing a fully interactive menu-driven interface for all system features.

CODE SNIPPETS

ORDER STATUS

```
public enum OrderStatus { no usages
    NEW, no usages
    PAID, no usages
    SHIPPED, no usages
    DELIVERED, no usages
    CANCELLED no usages
}
```

STOCK REDUCTION

```
public boolean reduceStock(int quantity) { no usages
    if (quantity <= 0) {
        throw new IllegalArgumentException(
            "Quantity to reduce must be greater than zero. Provided: " + quantity);
    }
    if (quantity > this.stockQuantity) {
        throw new IllegalStateException(
            "Insufficient stock for product '" + this.name + "'. " +
            "Requested: " + quantity + ", Available: " + this.stockQuantity);
    }
    this.stockQuantity -= quantity;
    return true;
}
```

PLACE ORDERS

```
public Order placeOrder(List<LineItem> items) { 1 usage
    if (items == null || items.isEmpty()) {
        throw new IllegalArgumentException(
            "Cannot place an order with a null or empty items list.");
    }

    for (LineItem item : items) {
        item.getProduct().reduceStock(item.getQuantity());
    }

    Order order = new Order( customer: this, items);
    return order;
}
```

STATUS UPDATE

```
public void updateStatus(OrderStatus newStatus) { no usages
    if (newStatus == null) {
        throw new IllegalArgumentException("New status cannot be null.");
    }
    if (this.status == OrderStatus.DELIVERED) {
        throw new IllegalStateException(
            "Cannot change the status of an already DELIVERED order.");
    }
    if (this.status == OrderStatus.CANCELLED) {
        throw new IllegalStateException(
            "Cannot change the status of an already CANCELLED order.");
    }
    if (newStatus == OrderStatus.CANCELLED) {
        this.status = OrderStatus.CANCELLED;
    }
    return;
}
```

PROCESS PAYMENT

```
public boolean processPayment(PaymentGateway paymentProvider) { no usages
    if (paymentProvider == null) {
        throw new IllegalArgumentException("PaymentGateway cannot be null.");
    }
    if (this.status != OrderStatus.NEW) {
        throw new IllegalStateException(
            "Payment can only be processed for orders in NEW status. " +
            "Current status: " + this.status);
    }
    boolean authorised = paymentProvider.authorizePayment( order: this);
    if (authorised) {
        this.status = OrderStatus.PAID;
    }
    return authorised;
}
```

PAYMENT AUTHORIZATION:

```
public class StripePaymentProvider implements PaymentGateway { no usages
    @Override 1 usage
    public boolean authorizePayment(Order order) {
        if (order == null) {
            throw new IllegalArgumentException(
                "[" + PROVIDER_NAME + "] Cannot authorise payment: Order must not be null.");
        }

        System.out.println "[" + PROVIDER_NAME + "] Initiating payment authorisation...";
        System.out.println "[" + PROVIDER_NAME + "] Order ID      : " + order.getId();
        System.out.println "[" + PROVIDER_NAME + "] Customer      : " + order.getCustomer().getName();
        System.out.println "[" + PROVIDER_NAME + "] Total Amount : $" +
            String.format("%.2f", order.getTotalAmount());
        System.out.println "[" + PROVIDER_NAME + "] Order Status : " + order.getStatus();

        // Decline: zero or negative charge amount
        if (order.getTotalAmount() <= 0) {
            System.out.println "[" + PROVIDER_NAME + "] DECLINED - " +
                "Charge amount must be greater than zero.";
            return false;
        }
    }
}
```

```
        // Decline: exceeds maximum single-charge limit
        if (order.getTotalAmount() > MAX_CHARGE_LIMIT) {
            System.out.println "[" + PROVIDER_NAME + "] DECLINED - " +
                "Charge of $" + String.format("%.2f", order.getTotalAmount()) +
                " exceeds the maximum single-charge limit of $" +
                String.format("%.2f", MAX_CHARGE_LIMIT) + ".";
            return false;
        }

        // Decline: order is not in a payable state
        if (order.getStatus() != OrderStatus.NEW) {
            System.out.println "[" + PROVIDER_NAME + "] DECLINED - " +
                "Order is not in NEW status. " +
                "Current status: " + order.getStatus() + ".";
            return false;
        }

        // All checks passed - authorise the charge
        System.out.println "[" + PROVIDER_NAME + "] AUTHORISED - " +
            "Payment of $" + String.format("%.2f", order.getTotalAmount()) +
            " successfully processed for order " + order.getId() + ".";
        return true;
    }
}
```


SAVE ORDERS:

```
@Override no usages
public void saveOrder(Order order) {
    if (order == null) {
        throw new IllegalArgumentException(
            "Cannot save a null Order.");
    }
    store.put(order.getOrderId(), order);
    System.out.println("[InMemoryOrderRepository] Order saved: " + order.getOrderId());
}
```

FIND ORDERS

```
@Override no usages
public Order findOrderById(UUID id) {
    if (id == null) {
        throw new IllegalArgumentException(
            "Cannot search for an order with a null ID.");
    }
    Order order = store.get(id);
    if (order == null) {
        throw new NoSuchElementException(
            "No order found with ID: " + id);
    }
    return order;
}
```

CREATE ORDERS:

```
public Order createOrder(Customer customer, List<LineItem> items) { no usages
    if (customer == null) {
        throw new IllegalArgumentException(
            "Customer must not be null when creating an order.");
    }
    if (items == null || items.isEmpty()) {
        throw new IllegalArgumentException(
            "Items list must not be null or empty when creating an order.");
    }
    Order order = customer.placeOrder(items);
    orderRepository.saveOrder(order);
    System.out.println("[OrderManager] New order created and saved: " + order.getOrderId());
    return order;
}
```

11. OUTPUT SCREENSHOTS

Main Menu

```
Catalogue seeded with 5 products.

=====

E-COMMERCE ORDER MANAGEMENT SYSTEM

=====

[1] Product Management
[2] Customer Management
[3] Order Management
[4] Payment Processing
[5] Update Order Status
[6] View Reports
[0] Exit

=====

Enter choice:
```

Product Management — Add Product

```
Enter choice: 1

=====

PRODUCT MANAGEMENT

=====

[1] Add New Product
[2] View All Products
[3] Restock a Product
[0] Back

=====

Enter choice: 1

-- Add New Product --
Product name       : Air Cooler
Description        : Make room coll
Price ($)          : 10000
Stock quantity     : 20
Product added: Air Cooler (ID: 1d5eb989-b40c-410f-8ee2-6d0dce1e4073)
```


Customer Registration

```
CUSTOMER MANAGEMENT

[1] Register New Customer
[2] View All Customers
[3] Update Customer Profile
[0] Back

Enter choice: 1

-- Register New Customer --
Full name : ABISHEIK
Email      : abisheikmister@gmail.com
Address    : H-Block hostel karpagam collge of engineering meyileripalaym othakalmandapam coimbatore
Customer registered: ABISHEIK (ID: a2841e90-3d47-4ee2-8ba2-59b527ad6b1f)
```

Place New Order with Item Selection

```
Enter choice: 3

ORDER MANAGEMENT

[1] Place New Order
[2] Add Item to Existing Order
[3] Cancel an Order
[4] View Order Details
[0] Back

Enter choice: 1

-- Place New Order --

#   Name                               Email                               Address
-----
1   ABISHEIK                           abisheikmister@gmail.com           H-Block hostel karpagam collge of engineering meyileripalaym othakalmandapam
    coimbatore

Select customer #: 1
```

```
1   Laptop PRO 15                       $1299.99   10
2   Wireless Mouse                      $29.99     50
3   4K Monitor 27"                      $499.99     5
4   Noise-Cancel Headset                 $199.99    20
5   Mechanical Keyboard                 $149.99    15
6   Air Cooler                          $10000.00  20

Select product # to add (0 to finish): 6
Quantity (available: 20): 1
Added: Air Cooler x1
Select product # to add (0 to finish): 0
[InMemoryOrderRepository] Order saved: 19e33001-b51e-4bd6-b125-53d389c2ef74
[OrderManager] New order created and saved: 19e33001-b51e-4bd6-b125-53d389c2ef74

=== Order Details ===
Order ID      : 19e33001-b51e-4bd6-b125-53d389c2ef74
Customer      : ABISHEIK (abisheikmister@gmail.com)
Order Date    : 2026-06-11T14:59:15.878046600
Status        : NEW
Items:
- Air Cooler | Qty: 1 | Unit Price: $10000.00 | Subtotal: $10000.00
```

Payment Processing — Stripe Authorisation

```
Enter choice: 4

-- Payment Processing (Stripe) --

#   Customer      Status   Total
-----
1   ABISHEIK      NEW      $10000.00

Select order # to pay: 1
Processing payment of $10000.00...
[Stripe] Initiating payment authorisation...
[Stripe] Order ID      : 19e33001-b51e-4bd6-b125-53d389c2ef74
[Stripe] Customer      : ABISHEIK
[Stripe] Total Amount   : $10000.00
[Stripe] Order Status   : NEW
[Stripe] AUTHORISED — Payment of $10000.00 successfully processed for order 19e33001-b51e-4bd6-b125-53d389c2ef74.
[InMemoryOrderRepository] Order saved: 19e33001-b51e-4bd6-b125-53d389c2ef74
[OrderManager] Payment successful. Order 19e33001-b51e-4bd6-b125-53d389c2ef74 is now PAID.
Payment authorised. Order is now PAID.
```

Order Status Update (PAID → SHIPPED → DELIVERED)

```
Enter choice: 5

-- Update Order Status --

#   Customer      Status   Total
-----
1   ABISHEIK      PAID      $10000.00

Select order #: 1
Current status: PAID
Available statuses:
[1] NEW
[2] PAID
[3] SHIPPED
[4] DELIVERED
[5] CANCELLED
Select new status #: 3
[InMemoryOrderRepository] Order saved: 19e33001-b51e-4bd6-b125-53d389c2ef74
[OrderManager] Order 19e33001-b51e-4bd6-b125-53d389c2ef74 status updated to: SHIPPED
Status updated to: SHIPPED
```

```
Enter choice: 5

-- Update Order Status --

#   Customer      Status   Total
-----
1   ABISHEIK      SHIPPED   $10000.00

Select order #: 1
Current status: SHIPPED
Available statuses:
[1] NEW
[2] PAID
[3] SHIPPED
[4] DELIVERED
[5] CANCELLED
Select new status #: 4
[InMemoryOrderRepository] Order saved: 19e33001-b51e-4bd6-b125-53d389c2ef74
[OrderManager] Order 19e33001-b51e-4bd6-b125-53d389c2ef74 status updated to: DELIVERED
Status updated to: DELIVERED
```

View Order Details

```
=== Order Details ===  
Order ID      : 19e33001-b51e-4bd6-b125-53d389c2ef74  
Customer      : ABISHEIK (abisheikmister@gmail.com)  
Order Date    : 2026-06-11T14:59:15.878046600  
Status        : NEW  
Items:  
  - Air Cooler | Qty: 1 | Unit Price: $10000.00 | Subtotal: $10000.00  
Total Amount: $10000.00  
=====
```

Filter Orders by Status

```
=====
```

```
Enter choice: 4  
  
Select status:  
[1] NEW  
[2] PAID  
[3] SHIPPED  
[4] DELIVERED  
[5] CANCELLED  
Choice: 4  
=== Order Details ===  
Order ID      : 19e33001-b51e-4bd6-b125-53d389c2ef74  
Customer      : ABISHEIK (abisheikmister@gmail.com)  
Order Date    : 2026-06-11T14:59:15.878046600  
Status        : DELIVERED  
Items:  
  - Air Cooler | Qty: 1 | Unit Price: $10000.00 | Subtotal: $10000.00  
Total Amount: $10000.00  
=====
```

```
=====
```

Order Summary with Grand Total

```
=====
VIEW REPORTS
=====

[1] All Products
[2] All Customers
[3] All Orders
[4] Orders by Status
[5] Order Summary (totals)
[0] Back
=====

Enter choice: 5

=====
No.      Customer      Status      Total
-----
1        ABISHEIK      DELIVERED   $10000.00
-----
GRAND TOTAL                                $10000.00
=====
```

Insufficient Stock Error Guard

```
#      Name      Price      Stock
-----
1      Laptop Pro 15      $1299.99      10
2      Wireless Mouse      $29.99      50
3      4K Monitor 27"      $499.99      5
4      Noise-Cancel Headset      $199.99      20
5      Mechanical Keyboard      $149.99      15
6      Air Cooler      $10000.00      19

Select product # to add (0 to finish): 6
Quantity (available: 19): 20
Added: Air Cooler x20

Select product # to add (0 to finish): 1
Quantity (available: 10): 11
Added: Laptop Pro 15 x11

Select product # to add (0 to finish): 0
Order failed: Insufficient stock for product 'Air Cooler'. Requested: 20, Available: 19
```

12. KEY OOP CONCEPTS DEMONSTRATED

- *Encapsulation* — All fields are private with controlled access via getters and setters
- *Interface & Abstraction* — PaymentGateway and OrderRepository are interfaces decoupling logic from implementation
- *Enumeration* — OrderStatus defines a strict, type-safe order lifecycle
- *Composition* — Order owns LineItems; LineItems cannot exist without an Order
- *Association* — Customer is linked to Orders; LineItem references a Product
- *Dependency Injection* — OrderManager receives OrderRepository through its constructor, not hardcoded
- *Exception Handling* — IllegalStateException and IllegalArgumentException guard all invalid operations
- *Design Pattern* — OrderManager acts as a Facade over the repository and order layers

RESULTS

The E-Commerce Order Management System was successfully developed and tested. All operations including product management, customer registration, order placement, payment processing, status lifecycle management, and order reporting work correctly using an in-memory repository. All validation guards for insufficient stock, invalid status transitions, and duplicate payments function as expected.

GITHUB REPOSITORY

Repository Link: [Click here-Source code link](#)

CONCLUSION

The project demonstrates the practical implementation of Java OOP concepts including encapsulation, interfaces, enumerations, composition, and dependency injection in a real-world e-commerce scenario. The system follows clean architecture principles by separating entity classes, service layers, repository interfaces, and the presentation layer. The menu-driven interface makes every feature accessible and testable interactively, reflecting how a production order management system would function.